AI-Powered Network Intrusion Detection & Alert Dashboard

Capstone Project Overview

Project Overview

This project builds a cybersecurity system that uses machine learning to detect suspicious network activity. The system analyzes network traffic data from the CICIDS2017 dataset, classifies it as benign or malicious, and displays alerts through a real-time dashboard built with Streamlit.

The goal is to demonstrate that a trained ML model can accurately identify common attack types and surface them in a way that a security analyst could act on.

Objectives

- Train and evaluate a machine learning model on labeled network traffic data
- Detect specific attack types: port scans, brute force attempts, DDoS, and web attacks
- Build a detection pipeline that processes new data and runs it against the trained model
- Display classified alerts in a Streamlit dashboard with filtering and visualizations
- Simulate at least two attack scenarios and demonstrate live detection
- Evaluate model performance using precision, recall, F1 score, and false positive rate

Dataset

CICIDS2017 from the Canadian Institute for Cybersecurity. This dataset contains labeled network flows covering benign traffic and multiple attack types including brute force, DoS/DDoS, port scanning, botnet, and web attacks. It includes 80+ extracted features per flow (packet counts, byte volumes, flow durations, flag counts, etc.).

Alternative datasets if CICIDS2017 proves too large or noisy: UNSW-NB15 or NSL-KDD. The team will finalize the dataset choice during Week 1.

System Architecture

The system has four components:

Layer	Description
1. Data Input	Load and parse the CICIDS2017 CSV files. Handle missing values, remove duplicates, and normalize features.
2. ML Model	Train a Random Forest classifier as the primary model. Compare against Logistic Regression and XGBoost. Select the best performer based on F1 score. Export the trained model using joblib.
3. Pipeline	A Python script that reads new data (simulated or replayed from the dataset), runs it through the same preprocessing steps, feeds it to the saved model, and outputs predictions with confidence scores.
4. Dashboard	A Streamlit web app that displays classified alerts, shows attack type breakdowns, timestamps, confidence levels, and allows filtering by severity and attack type.

Evaluation Metrics

The model will be evaluated on the following metrics across all attack classes:

Metric	Why It Matters
Precision	Of everything the model flagged as an attack, how much was actually an attack? High precision means fewer false alarms.
Recall	Of all actual attacks, how many did the model catch? High recall means fewer missed attacks.
F1 Score	The balance between precision and recall. This is the primary metric for model selection.
False Positive Rate	How often normal traffic gets flagged as malicious. Critical for real-world usability.
Confusion Matrix	Shows exactly where the model succeeds and fails across each attack category.

Team Roles

ML Developer (Cole)

Focus: Owns the ML model and the detection pipeline (heaviest coding role).

- Train, tune, and validate the ML model for intrusion detection
- Build and maintain preprocessing/feature pipeline used in both training and inference
- Export the final trained model (and any encoders/scalers) for the team to use
- Develop the detection/inference script that feeds new traffic data into the saved model
- Produce model outputs needed by the dashboard (prediction, attack label, confidence score)
- Own end-to-end code for model + pipeline, including fixes and performance improvements

Systems Integrator (Brian):

Focus: Build the Streamlit dashboard, integrate the ML pipeline outputs, and keep the project on track.

- Build the Streamlit dashboard (alerts table, filters, and key visualizations)
- Integrate the detection pipeline output into the frontend (load/display results, refresh behavior)
- Define and implement the interface/contract between model output and dashboard fields (timestamp, label, confidence, severity)
- Coordinate work across teammates (tasks, deadlines, integration check-ins)
- Perform end-to-end integration testing and troubleshoot issues between components
- Own project documentation coordination (status updates, README structure, and final deliverable packaging)

Research Lead (Frank):

Focus: Research, attack scenario planning, documentation, results analysis, and leading the final presentation.

- Research and select the dataset; document why it fits the project goals
- Define what counts as “suspicious behavior” and map it to labels/severity for the dashboard
- Plan the attack simulation scenarios (what to simulate, expected alerts, and success criteria)
- Own written documentation (project overview, methodology, assumptions, and limitations)
- Analyze model results (confusion matrix, false positives/negatives) and summarize findings
- Lead the final presentation and demo narrative (problem, approach, results, and takeaways)

Tech Stack

Tool	Purpose
Python	Primary language for all components
Pandas / NumPy	Data loading, cleaning, and manipulation
Scikit-learn	Model training, evaluation, and preprocessing pipelines
XGBoost	Gradient boosting model for comparison
Joblib	Save and load trained models
Streamlit	Dashboard frontend for alerts and visualizations
Matplotlib / Plotly	Charts and visual analytics in the dashboard
Scapy (optional)	Generate simulated attack traffic for live demo

Timeline

Week	Milestone	Owner
Week 1 (Apr 15 - 21)	All three finalize the dataset, set up the project repo, and define what attack types to detect.	Cole, Brian, Frank
Week 2 (Apr 22 - 28)	Cole explores and cleans the dataset. Frank researches each attack type in depth and documents what they look like in network traffic.	Cole, Frank
Week 3 (Apr 29 - May 5)	Cole performs feature engineering and trains the first baseline model. Frank writes the background section of the paper and defines what counts as a true positive for each attack category.	Cole, Frank
Week 4 (May 6 - 12)	Cole tunes and compares models. Frank designs the attack simulation test cases and creates the evaluation plan.	Cole, Frank
Week 5 (May 13 - 19)	Brian builds the detection pipeline. Frank begins building the presentation and writes the methodology documentation.	Brian, Frank
Week 6 (May 20 - 26)	Brian builds the Streamlit dashboard. Frank prepares the demo script and creates the evaluation criteria for the final results.	Brian, Frank
Week 7 (May 27 - Jun 2)	Brian connects the pipeline to the dashboard. Frank runs the attack simulations and records the results.	Brian, Frank
Week 8 (Jun 3 - 9)	All three test end-to-end. Cole generates final metrics. Frank analyzes results, compares them against the evaluation plan, and writes findings.	Cole, Brian, Frank
Week 9 (Jun 10 - 17)	All three finalize documentation, rehearse the demo, and present. Frank leads the presentation.	Cole, Brian, Frank

Expected Outcome

A working prototype that takes network traffic data, classifies it using a trained machine learning model, and surfaces alerts in a Streamlit dashboard. The demo will show the system detecting at least two simulated attack types with measurable accuracy, along with a full evaluation of model performance.

Deliverables

- Trained ML model exported as a joblib file
- Detection pipeline script that processes new data and outputs predictions
- Streamlit dashboard with alert display, filtering, and visualizations
- Model evaluation report with precision, recall, F1, and confusion matrix
- Live demo showing attack simulation and detection
- Final presentation